Лабораторна робота №4: Економічний аналіз та стратегічне планування програмного проєкту

Мета роботи: Оволодіти практичними навичками комплексного планування програмних проєктів на основі моделі COCOMO II та методології SLIM. Навчитися обґрунтовувати управлінські рішення щодо:

- економічної доцільності повторного використання коду (Reuse);
- стратегічного розподілу людських ресурсів та бюджету;
- мінімізації критичних ризиків та управління технічним боргом;
- математичного доведення неможливості стиснення графіку розробки («Impossible Region»).

<u>Програмна частина завдання:</u>

На основі отриманих розрахунків наведених завдань розробіть інструмент (обрати варіант 1 або варіант 2), який дозволяє їх автоматизувати.

<u>Технічні вимоги:</u>

- Розробити окремий модуль/бібліотеку, яка містить класи з відповідними математичними обрахунками.
- Реалізувати повну валідацію вхідних значень: Програма повинна блокувати введення некоректних даних та видавати попередження користувачу, якщо значення виходять за межі стандартів COCOMO II (AA має бути в діапазоні 0-8: SU: 0-50, DMCM, IM не можуть бути >100%;
- Продукт має підтримувати сценарій «What-if» аналізу: як зміниться фінальний Size, якщо параметр SU (розуміння коду) погіршиться на 20%? Результат порівняння вивести на екран.

Варіант 1 (max 4 бали): Розробка консольної утиліти «COCOMO II Calculator»

- Завдання: Створити консольний додаток (будь-яка мова програмування), який приймає необхідні параметри як аргументи командного рядка або через діалог.
- Після кожного введення програма перевіряє дані на допустимі межі, якщо дані некоректні – запитує повторно.
- Реалізувати можливість створення звіту у форматі Markdown, який містить фінальну таблицю розрахунків по кожному завданню. Це дозволяє PM-у миттєво отримати готовий документ для замовника.
- Результат: Програма не просто видає цифри, а генерує текстовий вердикт по кожному завданню.
    - Наприклад: Якщо AAF>100%, програма пише: "Увага! Адаптація недоцільна, пишіть код з нуля".
    - Або: Якщо бюджет перевищено, програма підсвічує це повідомленням "Project Overbudget!".

Варіант 2 (max 9 балів): Інтерактивний Dashboard «COCOMO II Calculator»

- Завдання: Створити GUI-застосунок (будь-яка технологія та фреймворк), що дозволяє керувати параметрами проєкту в реальному часі за допомогою повзунків (sliders).
- Реалізувати можливість збереження результатів розрахунків у файл (JSON або CSV).
- Візуалізація (Bar Chart): Реалізувати динамічну діаграму для проведення «What-if» аналізу за наступними сценаріями (за перемикачем):
    - Ефективність Reuse: Порівняння «номінального» обсягу коду (M+N) проти розрахованого «еквівалентного» обсягу (M+ESLOC). Це має наочно показати замовнику реальну економію (або її відсутність) від використання запозиченого коду.

- Структура та ліміти бюджету: Стовпчикова діаграма, що відображає накопичення вартості: Base Dev Cost → Total Budget (з урахуванням ризиків та техборгу) у порівнянні з лінією Граничного бюджету варіанту.
- Економіка ризиків: Порівняння показників RE_before та RE_after (Risk Exposure) для візуалізації доцільності витрат на контрзаходи (Cost_migration).
- Крива Патнама (SLIM): Візуалізація зростання трудомісткості (PM) при зміні повзунка термінів (TDEV) – наочна демонстрація «точки неможливості» для замовника.

- Результат: Інтерактивний інтерфейс, де при зміні будь-якого повзунка (наприклад, SU чи IM) графіки та фінальна вартість контракту перераховуються миттєво.

ІНФО-БЛОК

Пам'ятайте, що PM – показує, скільки годин ви купуєте у ринку, а AS – як ви розподіляєте ці години між працівниками в часі (TDEV).

1. Економічна сутність PM (Людино-місяця)

Показник PM (Effort) — це **загальний обсяг роботи**, необхідний для проєкту.

- Якщо проєкт потребує 10 PM, це означає, що потрібно оплатити 1520 годин роботи (10 × 152).
- Незалежно від того, чи ці години відпрацюють 2 людини за 5 місяців, чи 5 людей за 2 місяці — **сумарна вартість зарплатного фонду залишається незмінною**.

Формула вартості через зусилля:

$$Cost = PM \times 152 \times \text{Ставка}$$

Це найбільш точний спосіб розрахунку бюджету розробки, оскільки він базується на загальному обсязі витраченого часу.

2. Чому розрахунок через AS (кількість людей) може бути оманливим?

Показник AS — це **середня** кількість людей. На практиці в перший місяць може працювати 1 архітектор, а в піковий період — 10 розробників.
Якщо ми просто помножимо $AS \times TDEV \times \text{Ставка}$, ми отримаємо ту саму цифру, що і через PM, адже за визначенням $AS = PM/TDEV$.

Але є нюанс:
Якщо ви хочете ускладнити завдання (рівень Senior+), можна додати **"Вартість простою"** або **"Infrastructure Costs"**:

- Кожна людина (AS) потребує робочого місця, ліцензії на ПЗ та страховку (наприклад, +500/міснакожнуособузAS$).
- Тоді формула бюджету стане:

$$Total = (PM \times 152 \times \text{Ставка}) + (AS \times TDEV \times 500)$$

ПРАКТИЧНА ЧАСТИНА

Завдання 1. Розрахунок обсягу проєкту з урахуванням Reuse (ESLOC)

Контекст (Кейс): Ви — Project Manager у компанії «ElectroSoftware». Ви отримали замовлення на розробку деякого модуля для існуючого інтернет-магазину.

Замовник наполягає на максимальному використанні запозиченого коду (Reuse) для скорочення витрат – N рядків коду (ASLOC).

Ваші дії:

1) Обчисліть коефіцієнт адаптації (AAF) на основі параметрів адаптації згідно з аудитом техліда: AA (Оцінка та асиміляція), SU (Розуміння коду), DM (Зміна архітектури), CM (Зміна коду), IM (Інтеграція та тестування).
2) Розрахуйте еквівалентний обсяг (ESLOC) для запозиченого коду.
3) Визначте фінальний обсяг проєкту (Size у KLOC), якщо крім адаптованого коду, потрібно написати кількість M абсолютно нового коду.
4) Управлінський аналіз:
   - Порівняйте ESLOC із ASLOC та дайте відповідь замовнику: чи справді ми економимо N рядків коду? Який реальний відсоток економії зусиль ми отримуємо при використанні цього модуля (100% – AAF)?
   - Критичне запитання: Якщо AAF перевищить 100%, чи доцільно взагалі використовувати такий код? Обґрунтуйте свою позицію з точки зору вартості тестування (IM).

Формули для програмної реалізації:

1. $AAF = AA + SU + (0.4 \cdot DM) + (0.3 \cdot CM) + (0.3 \cdot IM)$
2. $ESLOC = N \cdot (AAF/100)$
3. $Size_{total} = M + ESLOC$

Таблиця індивідуальних варіантів (табл.1)

Кожен варіант містить унікальний бізнес-контекст (назву модуля), обсяги нового коду (M) та запозиченого (N), а також 5 параметрів адаптації для розрахунку за моделлю COCOMO II.

- M та N вказані в рядках коду (LOC).
- AA та SU – бали.
- DM, CM, IM – відсотки (при розрахунку AAF використовувати як числа від 0 до 100).

№	Назва модуля (Кейс)	Новий код M (LOC)	Reuse N (ASLOC)	AA	SU	DM%	CM%	IM%
1	Система авторизації (OAuth2)	12 000	4 500	2	15	10	10	30
2	Модуль обробки GPS-координат	8 500	6 200	4	35	25	20	45
3	Генератор PDF-звітів (Legacy)	7 000	5 800	6	45	30	40	60
4	Калькулятор знижок та акцій	10 500	3 400	3	20	15	15	25
5	Чат-бот підтримки користувачів	14 000	4 100	2	22	20	10	35
6	Інтеграція з платіжним шлюзом	9 200	5 000	5	30	20	25	50
7	Алгоритм пошуку по каталогу	11 000	3 900	3	28	15	20	40

8	Система кешування даних (Redis)	8 000	6 500	7	50	40	45	70
9	Модуль аналітики продажів	15 500	4 700	4	26	20	15	55
10	Валідатор форм та вводу даних	6 500	3 100	1	12	10	5	20
11	Система розсилки сповіщень	13 000	5 400	5	38	25	30	45
12	Експорт даних у CSV/Excel	10 000	4 200	3	18	15	10	30
13	Модуль керування кошиком	12 500	3 600	2	24	20	20	40
14	Обробник завантаження зображень	9 800	5 600	6	42	35	35	65
15	Система логування та аудиту	7 500	4 800	4	32	20	25	50
16	Модуль рекомендацій (AI-lite)	18 000	7 000	8	55	45	50	80
17	Сервіс порівняння товарів	11 200	3 300	3	20	15	15	35
18	API для мобільного додатка	14 500	5 200	4	28	25	20	45
19	Модуль обробки черги замовлень	9 000	6 000	5	40	30	35	60
20	Система відгуків та рейтингів	13 500	4 400	2	16	15	10	30

Завдання 2. Оцінка трудомісткості (Effort) та термінів (TDEV)

Використовуйте Post-Architecture модель COCOMO II для стратегічного планування ресурсів.

Вихідні дані:

- Розмір (Size): результат із Завдання 1 (перевести у KSLOC, тобто поділити на 1000).
- Множники EAF (Effort Adjustment Factor): RELY (Надійність), PCAP (Здібності Dev-ів), PCON (Стабільність кадрів), TOOL (Інструменти) – взяти з індивідуального варіанту, усі інші множники (CPLX, ACAP тощо) прийняти як Nominal (1.00).
- Коефіцієнт моделі (A): Прийняти A=2.94 (згідно зі стандартом COCOMO II).

Ваші дії:

1) Розрахуйте експоненту масштабу (E) на основі факторів масштабу (SF / Scale Factors): PREC (Наступність/Досвід подібних проєктів), FLEX (Гнучкість розробки), RESL (Архітектура та вирішення ризиків), TEAM (Згуртованість команди), PMAT (Зрілість процесів організації), які потрібно взяти з індивідуального варіанту.
2) Розрахуйте підсумковий EAF як добуток чотирьох індивідуальних множників.
3) Обчисліть трудомісткість (PM).
4) Розрахуйте календарний час (TDEV) за уточненою формулою для Post-Architecture.
5) Визначте середню чисельність персоналу (AS): скільки повних одиниць персоналу (людей) потрібно для завершення проєкту вчасно.

Формули для програмної реалізації:

1. Розрахунок Експоненти масштабу (E)

Це значення показує нелінійне зростання витрат при збільшенні розміру проєкту.

1. **Сума факторів масштабу (W):**

$$W = PREC + FLEX + RESL + TEAM + PMAT$$

(Значення беремо з таблиці SF).

2. **Експонента (E):**

$$E = 0.91 + 0.01 \cdot W$$

2. Розрахунок трудомісткості (PM)

$$PM = A \cdot (Size)^E \cdot EAF$$

- $A = 2.94$ (константа для моделі Post-Architecture).
- EAF — добуток усіх множників (RELY, PCAP, PCON, TOOL).
- Результат вимірюється у **людино-місяцях**.

3. Розрахунок календарного часу ($TDEV$)

$$TDEV = 3.67 \cdot (PM)^F$$

Де показник степеня F розраховується через експоненту E:

$$F = 0.28 + 0.2 \cdot (E - 0.91)$$

*(Результат вимірюється у **календарних місяцях**).*

.4. Середня чисельність персоналу (AS):

$$AS = \frac{PM}{TDEV}$$

Таблиця індивідуальних факторів масштабу SF (табл.2)

№	PREC	FLEX	RESL	TEAM	PMAT
1	Nom (3.72)	High (2.03)	V.High (1.41)	Nom (3.29)	High (1.56)
2	Low (4.96)	Nom (3.04)	Low (5.65)	Low (4.43)	Nom (3.12)
3	V.High (1.24)	V.High (1.01)	High (2.83)	V.High (1.10)	V.High (0.00)
4	Extra Low (6.20)	Low (4.05)	Nom (4.24)	Extra Low (5.48)	Low (4.68)
5	High (2.48)	Nom (3.04)	High (2.83)	High (2.19)	Nom (3.12)
6	Nom (3.72)	Low (4.05)	Low (5.65)	Nom (3.29)	Low (4.68)
7	Low (4.96)	Nom (3.04)	Nom (4.24)	Low (4.43)	Nom (3.12)
8	V.High (1.24)	High (2.03)	V.High (1.41)	V.High (1.10)	High (1.56)
9	Nom (3.72)	Nom (3.04)	High (2.83)	Nom (3.29)	Nom (3.12)
10	High (2.48)	V.High (1.01)	Nom (4.24)	High (2.19)	V.High (0.00)
11	Extra Low (6.20)	Extra Low (5.07)	Extra Low (7.07)	Extra Low (5.48)	Extra Low (7.80)
12	Nom (3.72)	High (2.03)	High (2.83)	Nom (3.29)	High (1.56)
13	Low (4.96)	Low (4.05)	Low (5.65)	Low (4.43)	Low (4.68)
14	V.High (1.24)	Nom (3.04)	V.High (1.41)	High (2.19)	V.High (0.00)
15	High (2.48)	High (2.03)	Nom (4.24)	V.High (1.10)	Nom (3.12)

16	Nom (3.72)	V.High (1.01)	Low (5.65)	Nom (3.29)	High (1.56)
17	Low (4.96)	Low (4.05)	High (2.83)	High (2.19)	Low (4.68)
18	High (2.48)	Nom (3.04)	V.High (1.41)	Nom (3.29)	High (1.56)
19	V.High (1.24)	High (2.03)	Nom (4.24)	Low (4.43)	Nom (3.12)
20	Nom (3.72)	Nom (3.04)	High (2.83)	High (2.19)	V.High (0.00)

Таблиця індивідуальних числових значень EAF (табл.3)

№	RELY (Надійність)	PCAP (Здібності)	PCON (Стабільність)	TOOL (Інструменти)
1	1.26 (V.High)	0.88 (High)	1.00 (Nom)	0.90 (High)
2	1.00 (Nom)	1.15 (Low)	1.29 (V.Low)	1.00 (Nom)
3	1.10 (High)	0.76 (V.High)	0.90 (High)	0.78 (V.High)
4	0.82 (Low)	1.00 (Nom)	1.00 (Nom)	1.17 (Low)
5	1.00 (Nom)	0.88 (High)	1.12 (Low)	0.90 (High)
6	1.26 (V.High)	1.00 (Nom)	1.00 (Nom)	1.00 (Nom)
7	1.10 (High)	1.15 (Low)	0.90 (High)	1.17 (Low)
8	1.00 (Nom)	0.76 (V.High)	0.81 (V.High)	0.90 (High)
9	1.26 (V.High)	0.88 (High)	1.12 (Low)	1.00 (Nom)
10	0.82 (Low)	1.00 (Nom)	0.90 (High)	0.90 (High)
11	1.00 (Nom)	1.15 (Low)	1.00 (Nom)	1.17 (Low)
12	1.10 (High)	0.88 (High)	0.90 (High)	1.00 (Nom)
13	1.26 (V.High)	1.00 (Nom)	1.12 (Low)	0.90 (High)
14	1.00 (Nom)	0.76 (V.High)	1.00 (Nom)	0.78 (V.High)
15	1.10 (High)	1.15 (Low)	1.29 (V.Low)	1.00 (Nom)
16	1.26 (V.High)	1.00 (Nom)	0.90 (High)	1.17 (Low)
17	0.82 (Low)	0.88 (High)	1.00 (Nom)	0.90 (High)
18	1.00 (Nom)	0.76 (V.High)	1.12 (Low)	1.00 (Nom)
19	1.10 (High)	1.00 (Nom)	1.00 (Nom)	0.90 (High)
20	1.26 (V.High)	0.88 (High)	0.90 (High)	0.78 (V.High)

Завдання 3. Управління ризиками (Risk Exposure)

Контекст: Під час планування ви ідентифікували критичний ризик, специфічний для вашого модуля (див. таблицю варіантів). Вам потрібно оцінити його вплив та обґрунтувати доцільність витрат на його запобігання.

Формули для розрахунку:

1) Risk Exposure (RE):

$$RE_before = P * L$$

$$RE_after = (P / 2) * L$$

де: P – ймовірність настання ризику (0.0 – 1.0), L – потенційні збитки (Loss) у робочих днях або грошах.

2) Risk Reduction Leverage (RRL) — доцільність контрзаходів:

$$RRL = \frac{RE_{before} - RE_{after}}{Cost_{mitigation}}$$

якщо RRL>1, то захід вважається економічно вигідним.

<u>Ваші дії:</u>

- Розрахуйте RE (Risk Exposure) для вашого кейсу згідно індивідуального варіанту та оцініть доцільність впровадження контрзаходу (Mitigation Strategy).

Таблиця індивідуальних варіантів (табл.4)

№	Опис ризику (Кейс)	Ймовірність P	Збитки L (днів)	Вартість запобігання Cost (днів)
1	Зміна API Google Maps під час релізу	0.4	20	5
2	Конфлікт ліцензій у Legacy-коді	0.6	15	3
3	Вихід ключового розробника (Senior)	0.3	40	8
4	Несумісність форматів даних Excel	0.5	12	4
5	Злам бази даних адрес (Security breach)	0.1	100	15
6	Відмова платіжного шлюзу (Downtime)	0.2	30	10
7	Низька швидкість пошуку (Performance)	0.7	10	6
8	Втрата даних у Redis при збої	0.25	25	5
9	Помилка у формулі аналітики (Audit fail)	0.15	50	12
10	Некоректна валідація (Data corruption)	0.45	18	4
11	Блокування поштових серверів (Spam)	0.35	22	6
12	Помилки при експорті великих файлів	0.5	14	5
13	Баг у логіці кошика (втрата замовлень)	0.1	80	20
14	Повільне завантаження зображень (UI)	0.6	8	4
15	Переповнення логів (Disk space full)	0.4	15	3
16	Модель рекомендацій дає 0 результатів	0.3	25	10
17	Помилка порівняння цін (Legal risk)	0.1	90	18
18	Несумісність API з версією Android	0.55	20	7
19	Затримка в черзі обробки (Latency)	0.4	12	5
20	Фейкові відгуки (Бот-атака)	0.65	10	4

Завдання 4. Економіка технічного боргу та рефакторингу

Сценарій: Ви як PM дозволили команді застосувати «швидке, але брудне» рішення (копіпаст з інших модулів вашого проєкту), щоб встигнути до маркетингового релізу. Це створило технічний борг. Тепер вам потрібно вирішити: чи виділяти ресурси на рефакторинг у наступному спринті, чи продовжувати жити з цим кодом.

Вихідні дані:

- Principal (Тіло боргу): P людино-годин – одноразові витрати на повне виправлення коду (рефакторинг).
- Interest (Відсотки): I – людино-годин – додатковий час, який команда витрачає щомісяця на підтримку, тестування та виправлення багів, спричинених цим «кривим» кодом.
- Ставка розробника: Staff_price = $X/година (взяти із табл.6).
- Горизонт планування: 12 місяців.

Ваші дії:

1. Розрахунок сумарної вартості боргу (Total Cost of Debt): Обчисліть витрати на борг за формулою:

$$TCD = P + (I \cdot n)$$

де n – кількість місяців (12).

2. Визначення точки беззбитковості (Break-even point): Розрахуйте, через скільки місяців витрати на «відсотки» зрівняються з витратами на «рефакторинг»:

$$n_{BEP} = \frac{P}{I}$$

3. Управлінське рішення:
   - Математичне обґрунтування: Якщо точка беззбитковості (n_вер) наступає раніше, ніж через 6 місяців, борг вважається «токсичним».
   - Бізнес-контекст: Дайте відповідь, чи варто проводити рефакторинг зараз, якщо компанія планує продати цей модуль іншому замовнику через 4 місяці? А якщо підтримка модуля планується на 2 роки?

Таблиця індивідуальних варіантів (табл.5)

№	Principal (P)	Interest (I)	№	Principal (P)	Interest (I)
1	30 год	5 год/міс	11	50 год	12 год/міс
2	45 год	4 год/міс	12	25 год	3 год/міс
3	60 год	15 год/міс	13	80 год	10 год/міс
4	20 год	8 год/міс	14	40 год	6 год/міс
5	35 год	7 год/міс	15	55 год	5 год/міс
6	50 год	5 год/міс	16	70 год	20 год/міс
7	25 год	10 год/міс	17	30 год	4 год/міс
8	40 год	12 год/міс	18	45 год	9 год/міс
9	65 год	6 год/міс	19	35 год	8 год/міс
10	15 год	5 год/міс	20	90 год	15 год/міс

Завдання 5. Розрахунок повної вартості проєкту та фінансове обґрунтування

Контекст: Ви підходите до етапу фіналізації бюджету для замовника. Вартість проєкту складається не лише з написання коду, а й з урахуванням ризиків та майбутніх витрат на підтримку.

Вихідні дані:

- Результат Завдання 2: Трудомісткість (PM) у людино-місяцях.
- Ставка розробника: Staff_price = $X/година.
- Норматив: 1 людино-місяць = 152 робочі години.
- Ризики: Витрати на запобігання ризику (Cost_migration) із Завдання 3.
- Техборг: Вартість рефакторингу (Principal) із Завдання 4.

Ваші дії:

1. Розрахунок базової вартості розробки (Cost_dev): Обчисліть вартість роботи команди без урахування додаткових факторів:

$$Cost_dev = PM * 152 * Staff_price$$

2. Формування повного бюджету проєкту (TotalBudget)

Ви як PM вирішуєте включити у фінальний кошторис витрати на мінімізацію критичного ризику та негайне усунення техборгу, щоб уникнути проблем у майбутньому.

$$TotalBudget = Cost_dev + Cost_M + Cost_P$$

де Cost_M = Cost_migration * 8 * Staff_price

Cost_P = Principal * Staff_price

Примітка: Для розрахунку ми приймаємо стандартний 8-годинний робочий день.

Студент має усвідомити, що реальна вартість проєкту – це не тільки зарплата за написання коду (Cost_dev), а й "страховий внесок" у безпеку (Cost_M) та інвестиція в якість архітектури (Cost_P).

3. Порівняльний аналіз:
   - Порівняйте отриманий TotalBudget із Граничним бюджетом вашого варіанту.
   - Якщо ви виходите за межі бюджету, обґрунтуйте: від чого ви відмовитесь (від рефакторингу чи від запобігання ризику) і які наслідки це матиме для стабільності модуля.

Приклад для перевірки (Варіант №1):

- Якщо $PM = 5.2$, то $Cost_{dev} = 5.2 \cdot 152 \cdot 30 = \$23,712$.
- Додаємо запобігання ризику (5 днів · 8 год · $30 = \$1,200$).
- Додаємо техборг (30 год · $30 = \$900$).
- **Разом:** $\$25,812$.
- **Вердикт:** Вкладаємось у ліміт $\$45,000$, проєкт є високоефективним.

Таблиця економічних показників (табл.6)

В умові ми використовуємо середню ставку (Blended Rate). Це середнє арифметичне між ставками Junior, Middle та Senior спеціалістів, які входять у команду.

№	Ставка розробника ($/год)	Граничний бюджет модуля ($)	Бізнес-контекст (рівень складності)
1	$35	$55,000	Середній аутсорс
2	$28	$48,000	Регіональна IT-компанія
3	$55	$120,000	Fintech-проєкт (High Security)
4	$25	$35,000	Стартап (MVP стадія)
5	$32	$50,000	Продуктова компанія (E-commerce)
6	$40	$75,000	Західний замовник (Enterprise)
7	$30	$52,000	Логістичний оператор
8	$45	$85,000	Cloud-native розробка
9	$38	$68,000	Аналітична платформа
10	$27	$32,000	Внутрішній проєкт автоматизації
11	$33	$62,000	Mobile App Backend
12	$42	$90,000	Big Data проект
13	$48	$105,000	Healthcare/Medtech модуль
14	$31	$45,000	Система управління складом
15	$34	$58,000	ERP-модуль для ритейлу
16	$60	$150,000	AI/ML інтеграція (Top Tier)
17	$29	$40,000	Портал державних послуг
18	$36	$70,000	EdTech платформа
19	$39	$65,000	Система моніторингу трафіку
20	$50	$115,000	Кібербезпека (CyberSec)

Завдання 6. Формування контрактної вартості та стратегічна комунікація

Контекст: Ви підготували зведений бюджет. Тепер ваша задача – презентувати його замовнику («ElectroSoftware»). Замовник бачить дві цифри: «Чисту розробку» (тільки код) та «Повний бюджет» (з ризиками та техборгом). Він схиляється до дешевшого варіанту.

<u>Ваші дії:</u>

1. Фіналізація розрахунків (TOTAL): Зведіть докупи всі витрати у фінальну таблицю для контракту:
   - Development Cost: PM * 152 * Staff_price.
   - Risk Migration Fund: Cost_M.
   - Technical Debt Refactoring: Cost_P.
   - TOTAL = Сума цих трьох компонентів.

2. Порівняльний аналіз ризиків (Business Case): Математично обґрунтуйте збитки від «дешевої розробки».
   - Покажіть замовнику, що якщо ризик настане (RE із Завдання 3), він втратить L днів, що у грошах дорівнює L*8*Staff_price.
   - Порівняйте цю суму із вартістю запобігання. Наприклад: "Витративши зараз $1200 на захист, ми страхуємо проєкт від прямого збитку у $24000.

3. Довгострокова перспектива техборгу:
   - Використовуючи дані із Завдання 4 (Interest), поясніть, що через 12 місяців «дешевий код» коштуватиме додаткові I*12*Staff_price.
   - Доведіть, що рефакторинг зараз (Principal) дешевше, ніж сума «відсотків» за рік.

4. Есе-аргументація (Звіт замовнику): Напишіть короткий текст, адресований стейкхолдеру:
   - Аргумент 1: Чому економія на ризиках – це гра в «рулетку» з грошима компанії.
   - Аргумент 2: Чому техборг – це «кредит під великі відсотки», який паралізує розвиток модуля через пів року.
   - Висновок: Чому ваша ціна (TotalBudget) є найбільш економічно вигідною, попри те, що вона не є найнижчою.

Приклад, як довести замовнику, що рефакторинг зараз (Principal) дешевше, ніж сума «відсотків» за рік.

Тут потрібно порівняти разову інвестицію (виправлення коду зараз) із постійними втратами (платою за помилку щомісяця).

Ось як це працює на цифрах:

1. Логіка розрахунку:

- **Сценарій А: Рефакторинг зараз (Principal).**
  Витрачаємо гроші один раз, щоб прибрати проблему назавжди.
  - $Principal = 30$ годин.
  - Вартість: 30 год $\times$ $35/год $= \mathbf{\$1,050}$.
- **Сценарій Б: "Життя з боргами" (Interest).**
  Ми не робимо рефакторинг, але щомісяця витрачаємо час на підтримку "кривого" коду.
  - $Interest = 5$ годин щомісяця.
  - За 12 місяців витрати часу: $5 \times 12 = 60$ годин.
  - Вартість за рік: 60 год $\times$ $35/год $= \mathbf{\$2,100}$.

2. Як сформулювати "Доказ" для замовника:

Пишемо математичне порівняння: Якщо ми зробимо рефакторинг зараз, це коштуватиме нам $1,050. Якщо ми цього не зробимо, то через складність коду ми будемо втрачати по 5 годин щомісяця. За рік ці втрати складуть $2,100. Отже, відкладаючи рефакторинг, ми фактично переплачуємо $1,050 (різниця між сценаріями) вже в перший рік роботи системи. Рефакторинг окупиться вже через 6 місяців (30/5=6 місяців).

Чому це важливо: Це ілюструє концепцію Точки беззбитковості (Break-even point).

- Якщо рефакторинг окупається за 3-6 місяців – це "токсичний" борг, його треба гасити негайно.

- Якщо рефакторинг окупається за 24 місяці, а проєкт триватиме лише 12 – PM може прийняти рішення не робити рефакторинг, бо це економічно не вигідно.

Завдання 7. Антикризове управління: Стиснення графіку (SLIM/Putnam Model)

Сценарій: Ви презентували замовнику розрахований у Завданні 2 термін розробки TDEV. Замовник категорично незадоволений: Ринок не чекає! Нам потрібно вийти в реліз удвічі швидше. Скоротіть термін розробки з TDEV до T_new = TDEV / 2 будь-якою ціною!

Ваші дії:

1. Математичне обґрунтування (Закон четвертого ступеня):

Згідно з моделлю Патнама, зусилля (PM) обернено пропорційні четвертому ступеню часу (TDEV). Використовуйте спрощену формулу для розрахунку зростання витрат:

$$\frac{PM_{new}}{PM_{old}} = \left(\frac{TDEV_{old}}{T_{new}}\right)^4$$

- Обчисліть, у скільки разів зросте трудомісткість (PM) при скороченні терміну вдвічі.
- Розрахуйте новий бюджет проєкту (TotalBudget_new) виходячи з отриманого значення PM_new.

2. Аналіз «Точки неможливості»:

У теорії управління проєктами існує «Impossible Region» (область неможливості). Навіть якщо найняти 100 розробників, вони не зможуть народити дитину за 1 місяць.

- Розрахуйте нову кількість персоналу: AS_new = PM_new / T_new.
- Дайте оцінку: чи зможе ваша компанія реально забезпечити таку кількість людей на один підмодуль?

3. Лист-відмова замовнику (PM Pitch): Напишіть коротку офіційну відповідь замовнику, де:
   - Аргумент 1: Наведіть розраховану суму бюджету при стислому графіку. Покажіть, що ціна зростає не вдвічі, а в 16 разів (2 в 4 степені).
   - Аргумент 2: Поясніть «Закон Брукса»: додавання людей у проєкт, що не вкладається в терміни, лише затягує його ще більше через складність комунікацій.
   - Компроміс: Запропонуйте альтернативу (наприклад, випустити лише критичний функціонал M_new у бажаний термін, відклавши Reuse та другорядні фічі.